

Full Length Article

rKAN: Rational Kolmogorov-Arnold networks

Alireza Afzal Aghaei ^a, Mehdi Hosseinzadeh ^{b,c,d,*}, Kourosh Parand ^{e,f,*}^a Centre for Research Impact & Outcome, Chitkara University Institute of Engineering and Technology, Chitkara University, Rajpura, 140401, Punjab, India^b School of Engineering and Technology, Duy Tan University, Da Nang, Vietnam^c Department of AI, School of Computer Science and Engineering, Galgotias University, Greater Noida, India^d Research Center of High Technologies and Innovative Engineering, Western Caspian University, Baku, Azerbaijan^e International Business University, Toronto, ON, Canada^f Department of Computer and Data Sciences, Faculty of Mathematical Sciences, Shahid Beheshti University, Tehran, Iran

ARTICLE INFO

Keywords:

Rational functions
Jacobi polynomials
Kolmogorov-Arnold networks
Physics-informed deep learning

ABSTRACT

The development of Kolmogorov-Arnold networks (KANs) marks a significant shift from traditional multi-layer perceptrons in deep learning. Initially, KANs employed learnable B-spline curves as their primary basis function, but their inherent complexity posed implementation challenges. Consequently, researchers have explored alternative basis functions such as wavelets, polynomials, and fractional functions. This paper explores the use of rational functions as a novel basis function for KANs. Two different approaches based on Padé approximation and rational Jacobi functions as trainable basis functions are proposed to define the rational KAN (rKAN). The rKAN's performance has been evaluated across various domains, including deep learning and physics-informed deep learning, where it demonstrates its superiority in comparison to its alternatives with 99.29% accuracy on MNIST classification, 86.6% accuracy in text sentiment analysis, and a solution to the CartPole reinforcement learning task in approximately 200 episodes.

1. Introduction

Neural networks have become essential tools in modern machine learning by contributing to advancements in a wide range of applications, such as regression, image classification, natural language processing, and scientific computation (Fares et al., 2025b). A key factor in their success is their capacity to approximate complex functions effectively. The selection of activation functions, which serve as basis functions, plays a critical role in determining the expressiveness and accuracy of their predictions. In contrast to traditional architectures, such as Multi-Layer Perceptrons (MLPs), which rely on fixed basis representations, the adaptive learning of these functions, as demonstrated by Kolmogorov-Arnold Networks (KANs) (Abd Elaziz et al., 2024; Fares et al., 2025a; Kiamari et al., 2024; Liu et al., 2025), leads to improved performance.

KANs represent a new class of neural network architectures that utilize compositions of one-dimensional functions as their primary operation, replacing conventional linear transformations and nonlinear activation functions. This design not only improves accuracy and function approximation capabilities but also enables the use of simple, tunable, one-dimensional functions as nonlinearities. Examples of such functions include splines, polynomials, Fourier, wavelets, and fractional functions (Somvanshi et al., 2024).

Each of these basis functions offers unique advantages, but their limitations often restrict their applicability to specific problem domains. Splines, for example, ensure smoothness and flexibility but are computationally inefficient and inherently local (Liu et al., 2025). Polynomial basis functions are straightforward and computationally efficient but suffer from instability issues like Runge's phenomenon, which limits their reliability for high-degree approximations (Boyd, 1992). Other specialized functions like Fourier series and wavelets are powerful for periodic data but may generalize poorly outside their specific problem domains (Bozorgasl & Chen, 2024). A common thread among these limitations is the difficulty in efficiently modeling functions with global behaviors, such as singularities or specific asymptotic trends.

Rational approximations, constructed as the quotient of two polynomials, directly address these challenges. They are particularly effective for functions with asymptotic behavior or singularities, and they naturally overcome the instability of high-degree polynomials while retaining computational efficiency (Babaei et al., 2024; Boullé et al., 2020; Sun & Yang, 2023; Tiller, 1983). Padé approximants are a specific type of rational approximation known for their high accuracy and have shown their value in fields like control theory and complex analysis (Baker & Gammel, 1961; Boyd, 2001). Despite this clear potential to solve the limitations of existing KAN basis functions, they have not yet been explored

* Corresponding authors.

E-mail addresses: alirezaafzalaghahi@gmail.com (A.A. Aghaei), mehdihosseinzadeh@duytan.edu.vn (M. Hosseinzadeh), k_parand@sbu.ac.ir (K. Parand).<https://doi.org/10.1016/j.neunet.2026.108888>

Received 21 August 2025; Received in revised form 30 December 2025; Accepted 21 March 2026

Available online 25 March 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

in the KAN framework. Our work integrates rational basis functions into KANs through two innovative methodologies:

- **Padé Approximation:** We construct rational functions as a ratio of two learnable Jacobi polynomial expansions. The core of this technique is the construction of learnable edge functions within the KAN, where each function is a Padé approximant built from the quotient of two trainable Jacobi polynomial series.
- **Rational Mapping of Jacobi Polynomials:** We apply a rational transformation to the input of Jacobi polynomials. This technique extends their domain from a finite interval to the entire real line and is well-suited for handling unbounded inputs. The approach also requires fewer parameters as well as hyperparameters and has a simpler evaluation, which results in faster training (Babaei et al., 2024).

The proposed method will be evaluated across various tasks, including regression, image and text classification, reinforcement learning, and solving differential equations. The results demonstrate performance improvements over traditional KANs and other baseline methods, especially in domains which require adaptability with asymptotic behavior.

The remainder of this paper is organized as follows. Section 2 reviews related work on rational functions in neural networks. Section 3 provides preliminaries to the work. Section 4 details the proposed methodologies, including the implementation of rational functions through Padé approximation and mapped Jacobi polynomials. Section 5 presents the experimental setup and results across various tasks. Finally, Section 6 concludes the paper with a discussion of the findings and directions for future research.

2. Related work

The use of rational functions in neural networks was first proposed by Leung and Haykin (Leung & Haykin, 1993) in 1993. They introduced the Rational Function Neural Network (RFNN), a feedforward architecture based on rational function approximations. RFNNs use linear adaptive filtering algorithms, like the recursive least squares algorithm. Such algorithms support efficient parameter estimation and global optimization without requiring nonconvex optimization techniques. Their approach demonstrated effectiveness in modeling nonlinear signal processing problems, including time series prediction and sea clutter modeling, with computational advantages over radial basis function networks (Leung & Haykin, 1993). However, their method constructs a single multivariate rational function from polynomial features of the input vector, whereas our approach composes univariate rational functions within the KAN architecture.

Helmke and Williamson (1995) extended the use of rational functions by connecting neural networks to linear system realization theory. They parametrized single-hidden-layer neural networks with continuous activation functions, and thus broadened network representations to rational functions. Their method is grounded in realization theory and established uniqueness and minimal dimensions under certain conditions, such as for linearly independent activation functions. The work also offered theoretical insights into rational neural network representations. However, their work provided a theoretical link between neural networks and rational functions from a systems theory perspective but does not propose a specific, trainable architecture for practical deep learning tasks.

Telgarsky (2017) explored the approximation capabilities of ReLU networks and rational functions. Using Newman polynomials, the author constructed rational functions to approximate ReLU activations, showing that these two classes can approximate each other efficiently, with representation sizes scaling polynomially in $\ln(1/\epsilon)$ rather than $1/\epsilon$. This work provided a foundation for incorporating rational functions as ReLU replacements in neural networks. While foundational, Telgarsky's focus was on the theoretical equivalence between the two function classes, rather than on designing a neural network where the rational

functions themselves are the core, trainable components, which is the central contribution of our research.

Boullé et al. (2020) introduced rational neural networks that use trainable rational functions as activation units. They demonstrated theoretically and empirically that these networks achieve superior approximation efficiency for smooth functions and require exponentially fewer nodes and layers than ReLU networks. Their experiments highlighted practical advantages in generative adversarial networks and Partial Differential Equation (PDE) solutions. The key distinction is that their work applies trainable rational activations in a standard MLP framework, whereas this paper integrates them as the fundamental basis functions on the edges of the KAN computational graph.

The success of rational functions in neural networks has been extended to solving engineering problems. For instance, Andrianova et al. (2021) developed a robust non-parametric identifier for singularly perturbed systems using rational differential neural networks, addressing multi-rate dynamics without prior knowledge of singular perturbation parameters. Similarly, Sun and Yang (2023) proposed rational function neural networks for rock physics modeling and used them to derive analytical models, such as Gassmann's equation, directly from field data. Although these application-specific models demonstrate the power of rational functions, they do not explore their use as a general-purpose, learnable basis within a KAN, which is the focus of our work.

Most recently, Yang et al. (2024) proposed a physics-informed rational neural network for modeling urban road traffic flow. Their method combines rational activation functions with noise-heavy-ball acceleration gradient descent to improve approximation accuracy and training efficiency. Applied to traffic modeling on a two-way single-lane road, their approach outperformed traditional methods in prediction error and computational efficiency, and introduced a new formulation for PDE-based traffic modeling. Like other prior works, their contribution lies in using rational functions as activation units within a PINN for a specific domain, which differs from our proposal to use them as the learnable basis functions in the general KAN architecture itself.

Despite these advances in applying rational functions to classical networks, their potential within the Kolmogorov-Arnold Network framework remains unexplored. To date, KAN research has relied on bases like B-splines or polynomials, which struggle with the very singularities and asymptotic behaviors that rational functions can model so effectively. This paper directly addresses this research gap by proposing Rational Kolmogorov-Arnold Networks (rKANs), the first architecture to integrate rational functions as the core learnable basis in a KAN. In the following section, we detail the necessary background and foundational concepts for this work.

3. Preliminaries

This section provides some necessary definitions for proposing rKAN.

3.1. Kolmogorov-Arnold networks

KANs are grounded in the Kolmogorov-Arnold representation theorem that asserts any multivariate continuous function can be decomposed into a finite sum of continuous univariate functions. The formal mathematical statement of this theorem is provided below.

Theorem 1. *For any continuous function $F : [0, 1]^v \rightarrow \mathbb{R}$, there exist continuous functions $\varphi_{q,k} : [0, 1] \rightarrow \mathbb{R}$ and continuous functions $\psi_k : \mathbb{R} \rightarrow \mathbb{R}$ such that*

$$F(\xi_1, \xi_2, \dots, \xi_v) = \sum_{k=1}^{2v+1} \psi_k \left(\sum_{q=1}^v \varphi_{q,k}(\xi_q) \right). \quad (1)$$

Proof. Intuitively, the theorem proposes that any complex, high-dimensional function can be represented by a “committee of experts.” For each of the $2v + 1$ experts, indexed by k , a set of inner functions $\varphi_{q,k}$

first transforms each input variable ξ_q into a new, one-dimensional representation. These transformations act independently on each input dimension, effectively re-scaling or warping them. Each expert then forms an intermediate judgment by simply summing these transformed inputs. This summation creates a set of simple, aggregated features. The outer functions ψ_k then map these aggregated features, the experts' initial judgments, to a final value. The theorem guarantees that this two-stage process of independent input transformations followed by transformations of their sums is sufficient to perfectly reconstruct any continuous multivariate function F by summing the outputs of all experts.

The formal proof is highly non-trivial and relies on advanced concepts in functional analysis. For more detailed definitions and proofs, refer to [Braun and Griebel \(2009\)](#), [Dzhenezher and Skopenkov \(2021\)](#), [Liu et al. \(2025\)](#), [Schmidt-Hieber \(2021\)](#). $\square$

Definition 1 (Kolmogorov-Arnold Network). A Kolmogorov-Arnold Network ([Liu et al., 2025](#)) is defined as a hierarchical composition of univariate function operators. Let $\mathcal{F}^{(0)} := \Xi = (\xi_1, \dots, \xi_v)$ be the vector of input identity functions. For each layer $l \in \{1, \dots, L-1\}$, define the transformation $\mathcal{T}_l : (\mathbb{R} \rightarrow \mathbb{R})^{d_{l-1}} \rightarrow (\mathbb{R} \rightarrow \mathbb{R})^{d_l}$ by

$$(\mathcal{T}_l(\mathcal{F}^{(l-1)}))_k := \sum_{j=1}^{d_{l-1}} \phi_{l,k,j} \circ f_j, \quad (2)$$

where $\mathcal{F}^{(l-1)} = (f_1, \dots, f_{d_{l-1}})$, and $\phi_{l,k,j} : \mathbb{R} \rightarrow \mathbb{R}$ are univariate functions. The network output is

$$F(\xi) := \sum_{k=1}^{d_{L-1}} \phi_{L-1,k} \circ f_k^{(L-2)}(\xi), \quad (3)$$

with $\mathcal{F}^{(L-2)} := \mathcal{T}_{L-2} \circ \dots \circ \mathcal{T}_1(\Xi)$. While the above defines the network as a composition of operators, its evaluation for a given input ξ is realized through a forward pass of numerical activations. This computational view is captured by the layer-wise transformation:

$$\hat{F}(\xi) = \Phi_{L-1} \circ \dots \circ \Phi_1 \circ \Phi_0(\xi), \quad (4)$$

where $\Phi_{q,k} = \phi_{q,k}(\cdot)$. Each layer l of the KAN performs a transformation defined by:

$$\xi^{l+1} = \Phi^l(\xi^l), \quad l = 0, 1, \dots, L. \quad (5)$$

In this formulation, the network has L layers. The input vector is $\xi \in \mathbb{R}^v$, where $v = d_0$. The term d_l denotes the width (number of neurons) of layer l . The function $\phi_{l,k,j}$ is a learnable univariate activation function on the edge connecting the j -th neuron in layer $l-1$ to the k -th neuron in layer l . The symbol $\circ$ represents function composition. Each layer Φ^l is a function vector that maps an input vector $\xi^l \in \mathbb{R}^{d_{l-1}}$ to an output vector $\xi^{l+1} \in \mathbb{R}^{d_l}$. The final output of the network, $\hat{F}(\xi)$, is a scalar approximation of the target function $F(\xi)$.

Originally, KANs were designed using spline functions to represent the univariate functions $\phi_{l,k,j}(\cdot)$. However, the use of splines introduces certain limitations, particularly with respect to computational efficiency, as they often result in high time complexity ([Liu et al., 2025](#)). To address these challenges, several alternative approaches have been proposed in recent literature. These include replacing splines with radial basis functions ([Li, 2024](#); [Ta, 2024](#)), wavelet-based representations ([Bozorgasl & Chen, 2024](#)), Chebyshev polynomials ([SS, 2024](#)), and fractional Jacobi functions ([Aghaei, 2025](#)). Among these options, both the Chebyshev KAN and its more advanced variant, the fractional Jacobi KAN, have demonstrated superior performance in terms of accuracy. The subsequent section explains Jacobi polynomials, which form the basis for constructing rKAN.

3.2. Jacobi polynomials

Jacobi polynomials arise in the study of orthogonal polynomials and have significant applications in numerical analysis, particularly for function approximation ([Rad et al., 2023](#)). These polynomials, denoted as

$J_n^{(\alpha,\beta)}(\xi)$ for $n = 0, 1, \dots$, are defined using the Gamma function:

$$J_n^{(\alpha,\beta)}(\xi) = \frac{\Gamma(\alpha + n + 1)}{n! \Gamma(\alpha + \beta + n + 1)} \sum_{m=0}^n \binom{n}{m} \frac{\Gamma(\alpha + \beta + n + m + 1)}{\Gamma(\alpha + m + 1)} \left(\frac{\xi - 1}{2}\right)^m, \quad (6)$$

where the parameters $\alpha, \beta > -1$ control the shape of the polynomial. The first few terms are:

$$\begin{aligned} J_0^{(\alpha,\beta)}(\xi) &= 1, \\ J_1^{(\alpha,\beta)}(\xi) &= \alpha + 1 + \left(1 + \frac{\alpha}{2} + \frac{\beta}{2}\right)(\xi - 1), \\ J_2^{(\alpha,\beta)}(\xi) &= \frac{(\alpha + 1)(\alpha + 2)}{2} + \frac{(\alpha + 2)(3 + \alpha + \beta)(\xi - 1)}{2} \\ &\quad + \frac{(3 + \alpha + \beta)(4 + \alpha + \beta)(\xi - 1)^2}{8}. \end{aligned} \quad (7)$$

Jacobi polynomials form an infinite sequence of orthogonal functions, defined using the standard Euclidean inner product:

$$\langle J_m^{(\alpha,\beta)}, J_n^{(\alpha,\beta)} \rangle_{\omega(\xi)} = \int_{-1}^1 J_m^{(\alpha,\beta)}(\xi) J_n^{(\alpha,\beta)}(\xi) \omega(\xi) d\xi = \langle J_m^{(\alpha,\beta)}, J_n^{(\alpha,\beta)} \rangle_{\omega(\xi)} \delta_{m,n}, \quad (8)$$

where $\omega(\xi) = (1 - \xi)^\alpha (1 + \xi)^\beta$ is a weight function and $\delta_{m,n}$ denotes the Kronecker delta. This orthogonality leads to several useful properties ([Shen et al., 2011](#)). For instance, Jacobi polynomials exhibit a symmetry property expressed by the relation $J_n^{(\alpha,\beta)}(-\xi) = (-1)^n J_n^{(\beta,\alpha)}(\xi)$, and their boundary behavior is characterized asymptotically by [Aghaei \(2025\)](#)

$$\begin{aligned} |J_n^{(\alpha,\beta)}(-1)| &\approx n^\beta \quad n \gg 1, \\ J_n^{(\alpha,\beta)}(+1) &\approx n^\alpha \quad n \gg 1. \end{aligned} \quad (9)$$

These polynomials can be generalized using the Gaussian hypergeometric function as

$$J_n^{(\alpha,\beta)}(\xi) = \frac{(\alpha + 1)_n}{n!} {}_2F_1 \left(-n, n + 1 + \alpha + \beta; \frac{1}{2}(1 - \xi) \right), \quad (10)$$

and can be reduced to classical orthogonal polynomials. For instance, when $(\alpha, \beta) = (0, 0)$, one obtains the Legendre polynomials $\mathcal{L}(\xi)$; for $(\alpha, \beta) = (-\frac{1}{2}, -\frac{1}{2})$, the Chebyshev polynomials of the first kind $\mathcal{T}(\xi)$ arise; when $(\alpha, \beta) = (\frac{1}{2}, \frac{1}{2})$, the second kind Chebyshev polynomials $\mathcal{U}(\xi)$ are obtained; and setting $(\alpha, \beta) = (\lambda, \lambda)$ yields the Gegenbauer polynomials $\mathcal{G}(\xi)$.

Employing the orthogonality property, the derivatives of Jacobi polynomials are expressible in terms of lower-degree terms of themselves with shifted parameters:

$$\frac{d^m}{d\xi^m} J_n^{(\alpha,\beta)}(\xi) = \frac{\Gamma(n + m + 1 + \alpha + \beta)}{2^m \Gamma(m + 1 + \alpha + \beta)} J_{n-m}^{(\alpha+m, \beta+m)}(\xi). \quad (11)$$

Furthermore, for a vector of Jacobi polynomials $\mathcal{J}^{(\alpha,\beta)} = [J_0^{(\alpha,\beta)}(\xi); \dots; J_n^{(\alpha,\beta)}(\xi)]$, the process of differentiation can be simplified into a matrix-vector product. We can define a lower triangular operational matrix for the derivative such that:

$$\frac{d^m}{d\xi^m} \mathcal{J}^{(\alpha,\beta)} = \mathfrak{D}^m \mathcal{J}^{(\alpha,\beta)}, \quad (12)$$

Here, $\mathfrak{D}^m$ is the m -th power of the single-derivative operational matrix $\mathfrak{D}$. Each element $\mathfrak{D}_{i,j}$ of this matrix represents a projection coefficient. Specifically, it quantifies how much of the j -th basis polynomial $J_j^{(\alpha,\beta)}$ is present in the derivative of the i -th basis polynomial, $\frac{d}{d\xi} J_i^{(\alpha,\beta)}(\xi)$. The elements are formally defined through the inner product:

$$\mathfrak{D}_{i,j} = \frac{\int_{-1}^1 \frac{d}{d\xi} \left\{ J_i^{(\alpha,\beta)}(\xi) \right\} J_j^{(\alpha,\beta)}(\xi) \omega(\xi) d\xi}{\int_{-1}^1 J_i^{(\alpha,\beta)}(\xi) J_j^{(\alpha,\beta)}(\xi) \omega(\xi) d\xi}. \quad (13)$$

The derivative of any order can thus be expressed directly through powers of $\mathfrak{D}$ acting on sampled values of $\mathcal{J}^{(\alpha,\beta)}$ which can be used for exact gradient computation during backpropagation.

4. Rational KAN

This section introduces two ways to build rational KANs. One based on the well-known Padé approximation and one with a rational form of Jacobi polynomials. The first approach involves dividing two polynomials, commonly referred to as the Padé approximation. Typically, Taylor polynomials are used as the basis functions for this method (Boullé et al., 2020). However, as the polynomial degree increases, this choice can lead to numerical instability, such as Runge's phenomenon. This issue is mitigated by employing orthogonal polynomials, which are defined as linear combinations of Taylor polynomials (Biswas et al., 2021). Orthogonal polynomials offer advantageous properties, including boundedness, straightforward derivatives, and a tunable structure (Aghaei, 2025), which makes them well-suited for Padé approximation (He et al., 2018). The second approach applies a rational mapping directly to orthogonal polynomials (Babaei et al., 2024). This method is generally more stable, requires fewer hyperparameters, and is a more recent development compared to the Padé approximation (Boyd, 2001). In the following, both constructions are defined.

The general formulation of a KAN layer replaces the linear transformation and fixed activation of an MLP with a sum of learnable univariate functions, $y_k = \sum_j \phi_{jk}(x_j)$, where ϕ_{jk} is a unique function for each input-output neuron pair. A rational KAN would therefore implement each ϕ_{jk} as an independent rational function, complete with its own set of learnable parameters (e.g., polynomial coefficients and Jacobi parameters). While this architecture offers maximum expressive power, it leads to a quadratic increase in parameters with the layer width ($input\ dim \times output\ dim$), which can be computationally prohibitive and difficult to train. To create a practical and parameter-efficient model that isolates the benefit of the rational basis itself, we propose a constrained yet powerful alternative. Here, we implement our rational functions as single, learnable activation layers that are shared across all neurons, analogous to how a standard ReLU or Tanh function is used. Our experimental results (see Section 5) reveal that this parameter-efficient approach is sufficient to achieve superior performance compared to baselines. The following subsections detail two methods for constructing these rational basis functions.

4.1. Padé rKAN

The Padé approximant of order $[p/q]$ for a function $f(\xi)$ is a rational function $R(\xi) = P(\xi)/Q(\xi)$, where $P(\xi)$ is a polynomial of degree at most p and $Q(\xi)$ is a polynomial of degree at most q . The coefficients of P and Q are determined such that the Taylor series expansion of $R(\xi)$ matches that of $f(\xi)$ up to the highest possible order, typically $O(\xi^{p+q+1})$.

Before defining Padé rKAN, we recall the rational neural network approach proposed in previous works. In most of the works, researchers utilized the Padé activation unit as Biswas et al. (2021), Boullé et al. (2020), Molina et al. (2019), Yang and Wang (2024):

$$A^{[p/q]}(\xi) = \frac{\sum_{i=0}^p \theta_i^e \xi^i}{\sum_{i=0}^q \theta_i^d \xi^i} \quad (14)$$

for some weights θ^e and θ^d , hyperparameters p and q , as the activation function of neurons. However, this technique has some issues, including numerical instability of the polynomial term ξ^i as the output of a hidden layer is unbounded. Employing orthogonal polynomials like Chebyshev, Hermite, and Laguerre is proposed to be a replacement for polynomials (Biswas et al., 2021) in rational neural networks. Among these options, Jacobi polynomials offer two distinct advantages. First, they exhibit robust numerical stability and boundedness within their domain (Eq. (9)), which mitigates issues like Runge's phenomenon. Second, their shape is controlled by two hyperparameters, α and β , which can be trained to adapt the basis functions to a specific dataset. Despite these benefits, tunable Jacobi polynomials have not been utilized for Padé approximation in either classical neural networks or KANs.

In this paper, we propose to set the basis functions of KANs in the form:

$$\phi(\xi; p, q) = \frac{\sum_{i=0}^p \theta_i^e \mathcal{J}_i^{(\alpha, \beta)}(\tanh(\xi))}{\sum_{i=0}^q \theta_i^d \mathcal{J}_i^{(\alpha, \beta)}(\tanh(\xi))}. \quad (15)$$

In this formulation, θ_i^e and θ_i^d represent the learnable coefficients for the numerator and denominator polynomials, respectively. The term $\mathcal{J}_i^{(\alpha, \beta)}$ denotes the Jacobi polynomial of degree i . Here, we employ the hyperbolic tangent on the input data to ensure that the input of Jacobi polynomials falls into their desired domain, which ensures properties such as boundedness (Eq. 9). We consider α, β to be trainable parameters of the network. Making α and β trainable provides additional flexibility, which allows the network to learn the optimal shape of the basis polynomials. This may improve fit and numerical stability compared to using fixed polynomials (e.g., Legendre or Chebyshev, which are special cases of Jacobi polynomials), although one can fix these parameters to any valid number. As these parameters have mathematical constraints and should be greater than -1 , we utilize the well-known ELU activation function (Clevert et al., 2015), which possesses the property $\text{ELU} : \mathbb{R} \rightarrow (-\kappa, \infty)$:

$$\text{ELU}(\xi; \kappa) = \begin{cases} \xi & \text{if } \xi > 0, \\ \kappa \times (e^\xi - 1) & \text{if } \xi \leq 0. \end{cases} \quad (16)$$

To ensure meaningful Jacobi functions, we set $\kappa = 1$ to map the learnable parameters to the domain $(-1, \infty)$. While other transformations could enforce this constraint, we select the shifted ELU for its simplicity and common use in deep learning.

4.2. Jacobi rKAN

The Jacobi polynomials are originally defined on the interval $[-1, 1]$, which limits their use in approximating functions across desired intervals. Consequently, researchers have developed techniques to extend their definition to a potentially infinite domain. This property can be reached using mapped Jacobi functions, presented in the following.

Definition 2 (Mapped Jacobi function). By applying an invertible mapping function $\varphi : \Omega \rightarrow [-1, 1]$ to the input of Jacobi polynomials, the mapped Jacobi functions can be generated as:

$$\mathcal{R}_n^{(\alpha, \beta)}(\xi) = \mathcal{J}_n^{(\alpha, \beta)}(\varphi(\xi)). \quad (17)$$

For a semi-infinite domain $\Omega = (0, \infty)$, three major options for $\varphi(\xi)$, with the hyperparameter $\iota > 0$, are available:

- Logarithmic mapping (Fig. 1(a)):

$$\varphi(\xi; \iota) = 2 \tanh\left(\frac{\xi}{\iota}\right) - 1, \quad (18)$$

- Algebraic mapping (Fig. 1(b)):

$$\varphi(\xi; \iota) = \frac{\xi - \iota}{\xi + \iota}, \quad (19)$$

- Exponential mapping (Fig. 1(c)):

$$\varphi(\xi; \iota) = 1 - 2\exp\left(-\frac{\xi}{\iota}\right). \quad (20)$$

Finally, for the infinite interval $\Omega = (-\infty, \infty)$, one can use a nonlinear mapping with $\iota > 0$ to generate mapped Jacobi functions:

- Logarithmic mapping (Fig. 2(a)):

$$\varphi(\xi; \iota) = \tanh\left(\frac{\xi}{\iota}\right), \quad (21)$$

- Algebraic mapping (Fig. 2(b)):

$$\varphi(\xi; \iota) = \frac{\xi}{\sqrt{\xi^2 + \iota^2}}. \quad (22)$$

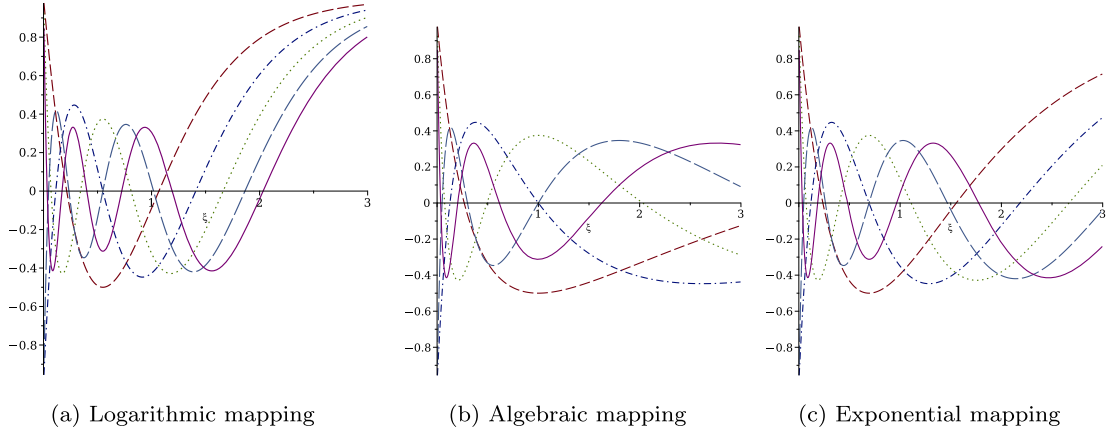


Fig. 1. Plots of mapped Jacobi functions $J_n^{(0,0)}(\xi)$ for $n = 2, 3, \dots, 6$ over semi-infinite.

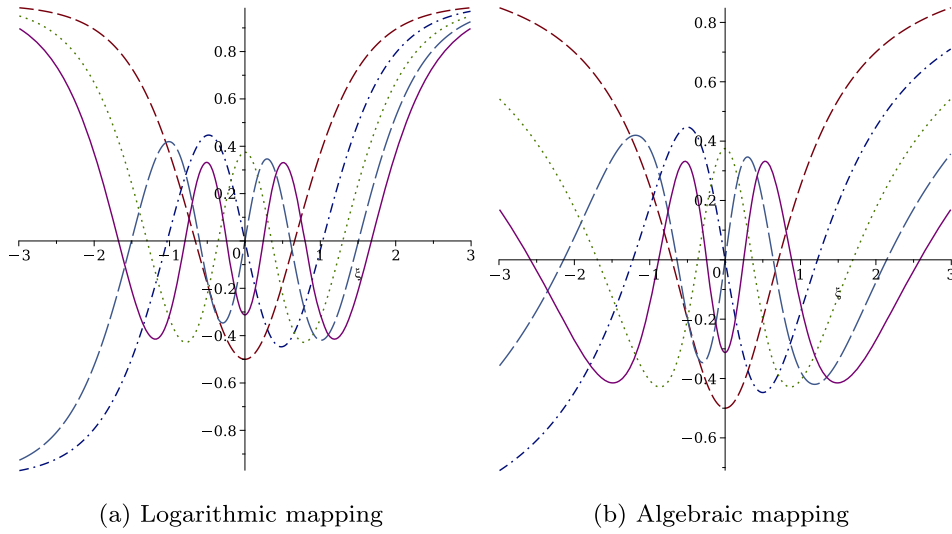


Fig. 2. Plots of mapped Jacobi functions $J_n^{(0,0)}(\xi)$ for $n = 2, 3, \dots, 6$ over infinite.

When the domain Ω is semi-infinite or infinite, it is common to call $\mathcal{R}_n^{(\alpha,\beta)}(\xi)$ rational Jacobi functions (Boyd, 2001). As illustrated in Fig. 1, these functions are non-zero, except for n real-valued distinct roots within their domain. For a more detailed discussion, we refer the reader to Aghaei et al. (2023), Boyd (2001), Rad et al. (2023).

Since the output of a network layer is unbounded, using an infinite mapping such as Eq. (22) or Eq. (21) can be beneficial for employing in KANs. As a result, the basis functions for Jacobi rKAN are defined as:

$$\phi(\xi; p) = J_p^{(\alpha,\beta)}(\varphi(\xi; \text{SoftPlus}(t))), \quad (23)$$

where the softplus function is defined as:

$$\begin{aligned} \text{SoftPlus} : \mathbb{R} &\rightarrow (0, \infty), \\ \text{SoftPlus}(t) &= \log(1 + \exp(t)), \end{aligned} \quad (24)$$

and is applied to the trainable parameter t to ensure its positivity.

4.3. Rational KAN layer

In a general rKAN layer, each basis function ϕ_{jk} is an independent rational function. This means that for a layer with d_{in} inputs and d_{out} outputs, there are $d_{\text{in}} \times d_{\text{out}}$ distinct rational functions, each with its own set of learnable parameters. The specific parameterization for each edge function ϕ_{jk} depends on the chosen rational form. For a Jacobi rKAN layer, each function ϕ_{jk} would be a mapped Jacobi polynomial with its own trainable parameters. The activation on the edge (j, k) would be

$J_n^{(\alpha_{jk}, \beta_{jk})}(\varphi(x_j; t_{jk}))$, where the shape parameters $(\alpha_{jk}, \beta_{jk})$ and the mapping parameter t_{jk} are unique to that specific connection. Similarly, for a Padé rKAN layer, each function ϕ_{jk} would be a Padé approximant formed by the ratio of two Jacobi polynomial expansions. The function would be:

$$\phi_{jk}(x_j) = \frac{\sum_{i=0}^p w_{p,ijk} J_i^{(\alpha_{p,jk}, \beta_{p,jk})}(\tanh(x_j))}{\sum_{i=0}^q w_{q,ijk} J_i^{(\alpha_{q,jk}, \beta_{q,jk})}(\tanh(x_j))}. \quad (25)$$

Here, polynomial weights $w_{p,jk}$, $w_{q,jk}$, and the Jacobi shape parameters are learned independently for every edge (j, k) .

5. Experiments

In this section, we evaluate the proposed rational KAN on various real-world and engineering tasks. Based on our experiment, employing only a single rational basis function $\phi(\cdot)$ as the activation function of a network can obtain superior results in comparison to alternative approaches, hence for all of the simulations we consider this approach. All experiments are implemented in Python using the PyTorch and TensorFlow libraries. The experiments are conducted on a PC equipped with an Intel Core i3-10100 processor, an NVIDIA GeForce GTX 1650

graphics card, and 24 GB of RAM. The implementation of this approach is publicly available on GitHub¹.

5.1. Deep learning tasks

This section presents classification and regression tasks simulated using rKAN.

5.1.1. Regression tasks

Analysis. We begin the assessment of rKAN with a regression task using synthetic data generated from three different functions with asymptotic behavior. These functions are defined as follows:

$$F_1(\xi) = \frac{\xi}{1 + \xi^2}; \quad F_2(\xi) = \frac{1}{1 + \xi^2}; \quad F_3(\xi) = \exp(-\xi^2). \quad (26)$$

For training, we sample 200 random points, and for testing, 100 random points within the interval $[-10, 10]$. To ensure a fair comparison, all models, including the baselines, use an identical network structure. A random search over simple configurations (one to five hidden layers with fewer than 50 neurons each) on a validation set yielded a $[1, 10, 20, 1]$ architecture for all experiments. A baseline MLP with Tanh or ReLU with 271 parameters is considered for comparison. The original KAN requires fewer than 300 parameters, and our rKAN architectures contain 280–300 parameters, depending on the rational function type and polynomial degree. In these experiments, the rKAN models were implemented by setting the activation functions in the hidden layers to either our proposed Jacobi-rKAN or Padé-rKAN, with the specific variant noted in the corresponding tables. The network is optimized using the L-BFGS optimizer with full batch processing for 50 epochs. We use the mean squared error (MSE) to evaluate both the training and testing accuracy. The MSE results for the test data are presented in Tables 1, 2, and 3, where p denotes the numerator degree for Padé-rKAN and the polynomial degree for Jacobi-rKAN. In all tables, we have employed (22) as the rational mapping of Jacobi functions. To provide a clear comparison against baseline methods, the performance of the best-performing rKAN configuration for each task is summarized in Table 4. It is seen that the rKAN is superior in comparison to fractional KANs as well as classical activation functions like hyperbolic tangent and ReLU. The results also highlight the sensitivity of the Padé-rKAN to the choice of polynomial degrees. The practical implication of this sensitivity is that the degrees of the numerator (p) and denominator (q) in Padé-rKAN are critical hyperparameters that require careful tuning. As seen in Table 1, a suboptimal choice (e.g., Padé[3/3]-rKAN) can lead to poor performance, whereas a well-chosen configuration (e.g., Padé[4/3]-rKAN) can yield state-of-the-art results. For practical applications, selecting these degrees involves a trade-off. Lower-degree polynomials are generally more stable and computationally faster but may lack the expressive power to capture complex functions. Higher degrees can improve accuracy but risk overfitting and increase training time. A common strategy for choosing optimal values is to perform a grid search over a small range of integers for p and q (e.g., from 2 to 6) on a validation set to find the best balance between model complexity and performance for a specific task.

It is also worth noting the poor performance of the original spline-based KAN (2.240×10^{-2} MSE for $F_1(\xi)$). This is likely because B-splines, which are piecewise polynomials defined on a finite grid, are inherently ill-suited to approximating functions with singularity or global asymptotic behavior, such as those in this example. They struggle to generalize the approximation without a significant increase in parameters and grid resolution. In contrast, rational functions are naturally adept at modeling such behavior.

An important distinction between our two proposed methods lies in how they handle the input domain. While both architectures can accept unbounded inputs, Padé-rKAN relies on the tanh function to compress the domain, whereas Jacobi-rKAN uses a scalable rational mapping as

Table 1

The MSE between the predicted values and the exact values for the $F_1(\xi)$. The best result is shown in bold format, and the second-best value in each column is underscored.

Model	$p = 2$	$p = 3$	$p = 4$	$p = 5$	$p = 6$
Jacobi-rKAN	4.223×10^{-7}	4.289×10^{-7}	3.273×10^{-6}	4.861×10^{-5}	1.132×10^{-4}
Padé[p/3]-rKAN	2.616×10^{-7}	2.778×10^{-2}	1.634×10^{-7}	7.760×10^{-7}	2.142×10^{-6}
Padé[p/4]-rKAN	2.951×10^{-5}	<u>1.075×10^{-6}</u>	<u>1.109×10^{-6}</u>	<u>9.297×10^{-7}</u>	<u>1.722×10^{-5}</u>
Padé[p/5]-rKAN	4.334×10^{-3}	3.930×10^{-3}	1.042×10^{-4}	1.140×10^{-3}	9.587×10^{-4}

Table 2

The MSE between the predicted values and the exact values for the $F_2(\xi)$.

Model	$p = 2$	$p = 3$	$p = 4$	$p = 5$	$p = 6$
Jacobi-rKAN	1.889×10^{-7}	8.220×10^{-7}	1.446×10^{-6}	9.917×10^{-6}	7.252×10^{-5}
Padé[p/3]-rKAN	<u>2.560×10^{-7}</u>	3.752×10^{-7}	7.343×10^{-7}	7.138×10^{-7}	2.728×10^{-6}
Padé[p/4]-rKAN	9.726×10^{-4}	6.778×10^{-7}	1.521×10^{-6}	<u>1.727×10^{-6}</u>	6.301×10^{-4}
Padé[p/5]-rKAN	4.420×10^{-3}	3.629×10^{-3}	6.551×10^{-4}	1.813×10^{-3}	2.964×10^{-4}

Table 3

The MSE between the predicted values and the exact values for the $F_3(\xi)$.

Model	$p = 2$	$p = 3$	$p = 4$	$p = 5$	$p = 6$
Jacobi-rKAN	5.330×10^{-7}	<u>5.350×10^{-7}</u>	3.540×10^{-6}	1.730×10^{-5}	2.760×10^{-4}
Padé[p/3]-rKAN	2.590×10^{-7}	1.100×10^{-2}	5.670×10^{-7}	7.750×10^{-7}	2.680×10^{-6}
Padé[p/4]-rKAN	1.460×10^{-6}	5.250×10^{-7}	6.990×10^{-7}	9.890×10^{-2}	<u>4.550×10^{-5}</u>
Padé[p/5]-rKAN	4.070×10^{-3}	4.420×10^{-3}	4.340×10^{-6}	<u>1.290×10^{-5}</u>	1.240×10^{-3}

defined in Eq. (22). To evaluate the practical implications of this difference, we conducted an experiment to assess each model's robustness to an increasing input domain width. We used the same network architecture and functions (F_1, F_2, F_3) from the previous section, but varied the input domain from a narrow range (e.g., $[-10, 10]$) to a very wide one (e.g., $[-2000, 2000]$). The results, shown in Fig. 3, reveal a critical performance difference. For the Padé-rKAN, the MSE consistently increases as the domain widens. This performance degradation is likely caused by the saturating nature of the tanh function, which loses discriminative power by mapping all large-magnitude inputs to values very close to ± 1 . In contrast, the Jacobi-rKAN demonstrates remarkable stability, maintaining a low MSE even on extremely wide domains. As a result, this behavior empirically validates that the rational mapping used in Jacobi-rKAN is more robust and better suited for problems where the input data is not confined to a small interval.

A key criticism of KANs is their computational cost. To address this, we analyzed the training time of our rKAN models as a function of their primary complexity parameter: the polynomial degree. Table 5 reports the mean time per L-BFGS training step for various model configurations. As expected, the training time for both Jacobi-rKAN and Padé-rKAN increases with the polynomial degree. The Padé-rKAN is computationally more expensive than the Jacobi-rKAN for a comparable total degree, as it requires the evaluation of two separate polynomial expansions (numerator and denominator). It is seen that higher-degree polynomials can offer greater expressive power and lower error (as seen in Tables 1, 2, 3) but at the cost of increased training time.

Comparison to other KANs. To compare our proposed method with other KAN architectures, we conducted a benchmark on a two-dimensional regression task, where the goal was to approximate the function $f(x_1, x_2) = \exp(-x_1^2) \cos(x_2)$ over the domain $[-5, 5]^2$. All models were based on a shared network width of $[2, 4, 1]$ and trained for 20 epochs using the L-BFGS optimizer. We designed the comparison to evaluate different basis functions by varying their internal hyperparameters. For polynomial-based models, including both of our rKAN models, fKAN (Aghaei, 2025), and ChebKAN (SS, 2024), we varied the polynomial degree from 2 to 6. For the original spline-based KAN (Liu et al., 2025), we varied the spline order k from 2 to 6, and for wavelet KAN (wav-KAN) (Bozorgasl & Chen, 2024), we tested several standard mother wavelets,

¹ <https://github.com/alirezaafzalaghahi/rKAN>

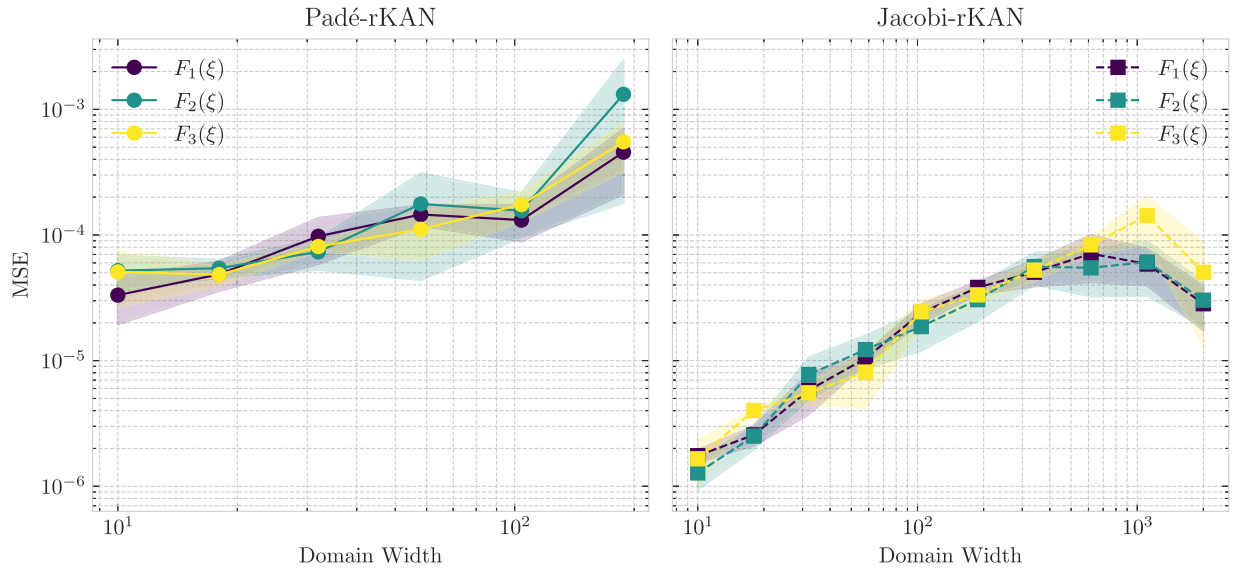


Fig. 3. Performance comparison of Padé-rKAN and Jacobi-rKAN as a function of input domain width. The MSE is plotted on a log scale for three regression tasks (F_1 , F_2 , F_3). The Jacobi-rKAN demonstrates superior robustness and maintains low error on wide domains, while the Padé-rKAN's performance degrades significantly.

Table 4

Comparison of the best-performing rKAN model against baseline architectures on the three synthetic regression tasks. Values represent the MSE on the test set. The proposed rKAN model achieves the best performance on all tasks, with the best result for each task highlighted in bold.

Model	$F_1(\xi)$	$F_2(\xi)$	$F_3(\xi)$
rKAN (Best)	1.634×10^{-7}	1.889×10^{-7}	2.590×10^{-7}
fKAN (Best)	3.330×10^{-7}	4.629×10^{-7}	4.320×10^{-7}
KAN (Spline)	2.240×10^{-2}	1.520×10^{-2}	1.890×10^{-2}
MLP (Tanh)	2.711×10^{-7}	2.963×10^{-7}	1.490×10^{-6}
MLP (ReLU)	5.143×10^{-4}	3.402×10^{-5}	4.750×10^{-5}

Table 5

Analysis of computational cost. The table shows the mean time per L-BFGS training step (in milliseconds) for different rKAN configurations with shape [1, 10, 20, 1] as well as the KAN (Liu et al., 2025) with a grid size of 5.

Jacobi-rKAN(p)		Padé-rKAN[p/q]		KAN	
Degree (p)	Time (ms)	Degrees [p/q]	Time (ms)	Degree (k)	Time (ms)
2	9.62	[2/3]	11.19	1	195.1
3	11.54	[2/4]	20.24	2	239.9
4	14.57	[2/5]	40.73	3	259.0
5	24.58	[3/4]	48.95	4	338.0
5	24.58	[3/5]	51.49	5	374.8

including Mexican Hat, Morlet, and Shannon. We also included other recent variants like FastKAN (Li, 2024) and fKAN (Zhangyanbo, 2024) in the benchmark. Fig. 4 presents a Pareto analysis of the results. The plot shows model accuracy against parameter count, with training time as a visual variable. For instance, ChebKAN(6) achieves the lowest MAE of 1.20×10^{-1} with 61 parameters, and fKAN reaches a comparable MAE of 1.21×10^{-1} but requires 3468 parameters. Our proposed rKAN(6) model achieves the third-best accuracy with a MAE of 1.26×10^{-1} using only 20 parameters, which demonstrates high parameter efficiency. The original KAN models, while accurate (e.g., KAN(2) with MAE of 1.27×10^{-1}), are slower and require significantly more parameters (188).

5.1.2. Image classification

Recent studies have explored the application of KANs in various image processing tasks, such as image classification (Aghaei, 2025; Azam

& Akhtar, 2024; Cheon, 2024; Seydi, 2024), image denoising (Aghaei, 2025), and image segmentation (Li et al., 2024). In this section, we focus on the classification task using some benchmark datasets, including MNIST, CIFAR10, CIFAR100, and SVHN. We designed a 2-dimensional convolutional neural network for this task, as illustrated in Fig. 5. The network processes the input through two sequential convolutional blocks. The first block consists of a convolutional layer with 32 filters (3×3 kernel) followed by a Jacobi-rKAN activation function and a max-pooling layer. The second block is similar but uses 64 filters. Finally, the output is flattened and passed through a dropout layer with a rate 0.5 and a dense softmax layer for classification. The network was trained using the Adam optimizer with the learning rate 0.001, a batch size of 512, and 30 epochs. The validation loss and accuracy during training are depicted in Fig. 6. Additionally, the performance metrics on the test set are presented in Table 6, which also compares the results with those obtained using fractional KAN (Aghaei, 2025) and common activation functions like hyperbolic tangent and ReLU. The learning curves of the proposed networks are compared in Fig. 7. We have also compared the proposed method with FastKAN, wav-KAN, and the original KAN architecture with a comparable number of trainable parameters. Based on our experiments, these KAN-based architectures, especially the spline KAN, often benefit from larger batch sizes for optimal performance. However, to maintain consistency across all experiments, we used a batch size of 512 for all models; consequently, the results are much less accurate than those of our proposed method.

5.1.3. Text classification

In this section, we evaluate the performance of various networks on a text classification task, specifically sentiment analysis of the IMDB dataset. The objective is to classify movie reviews as either positive or negative. To accomplish this, we implemented a 1D CNN similar to fKAN (Aghaei, 2025). The model processes the text using an embedding layer, followed by two 1D convolutional layers and a global max pooling 1D layer. The empirical results of our comparative analysis are presented in Table 7 using different binary classification metrics. To provide a more comprehensive benchmark, we expanded our comparison to include not only fKAN but also several other prominent KAN architectures: the original spline-based KAN, wav-KAN, and FastKAN. For these KAN-based models, the text was processed by an embedding layer followed by the KAN, with a comparable number of parameters to ensure a fair comparison. The empirical results of this analysis are presented in Table 7.

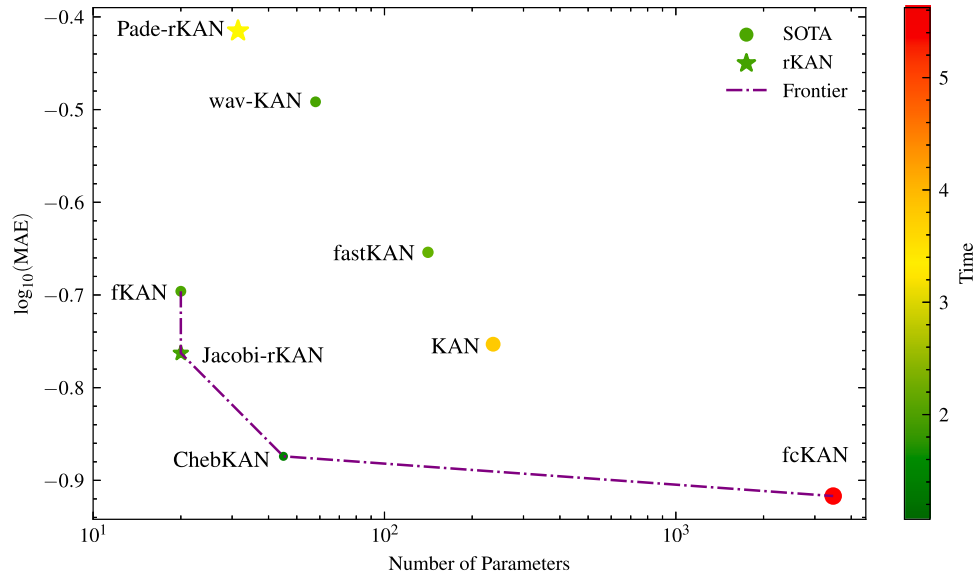


Fig. 4. A comparison between different KAN architectures for solving a regression task, illustrated as a scatter plot of performance versus model size. The y-axis represents the Average MSE on a logarithmic scale, while the x-axis shows the number of parameters. Both color and size denote the required CPU time to run the model.

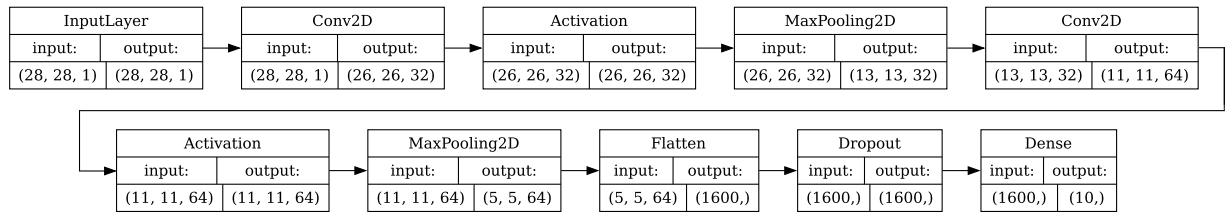


Fig. 5. The architecture of the proposed method for image classification, which is based on the architecture chosen in [Aghaei \(2025\)](#) to ensure a fair comparison.

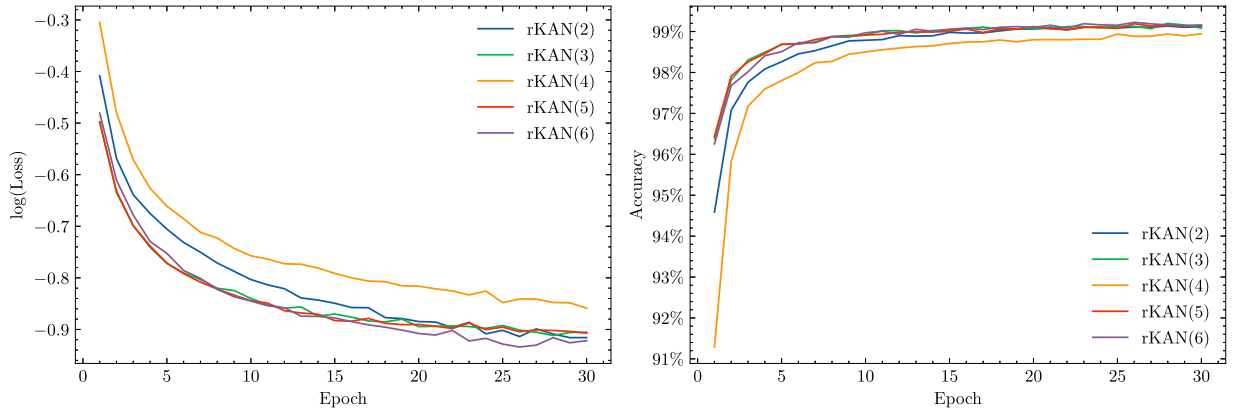


Fig. 6. Loss and accuracy of MNIST classification using Jacobi-rKAN with different values of Jacobi orders.

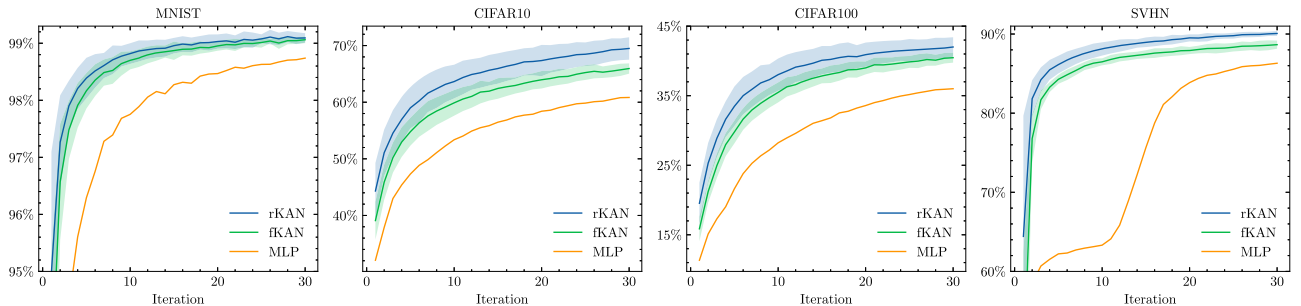


Fig. 7. Learning curves of rKAN in comparison to fKAN and MLP network with hyperbolic tangent activation function in solving image classification tasks. Results are averaged over 5 independent runs with different random seeds from 42 to 46.

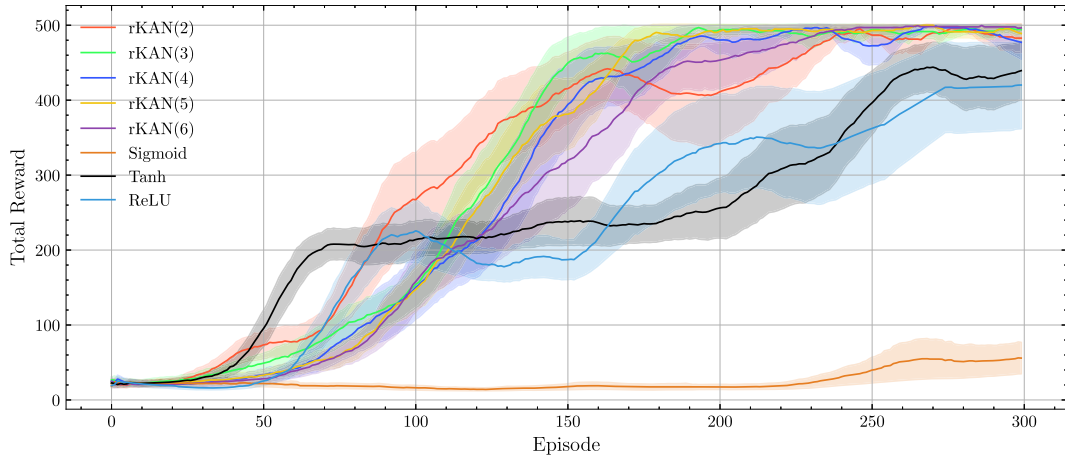


Fig. 8. Reward trajectories across training episodes for DQN agents with different activation functions on CartPole-v1. Each curve shows the episode-wise mean return and its 70% confidence interval over 10 independent runs with random seeds 40 to 49, smoothed with a 20-episode moving average. It is seen that the Jacobi-based rKAN activations exhibit faster convergence and higher final performance compared to classical activations such as ReLU and Tanh.

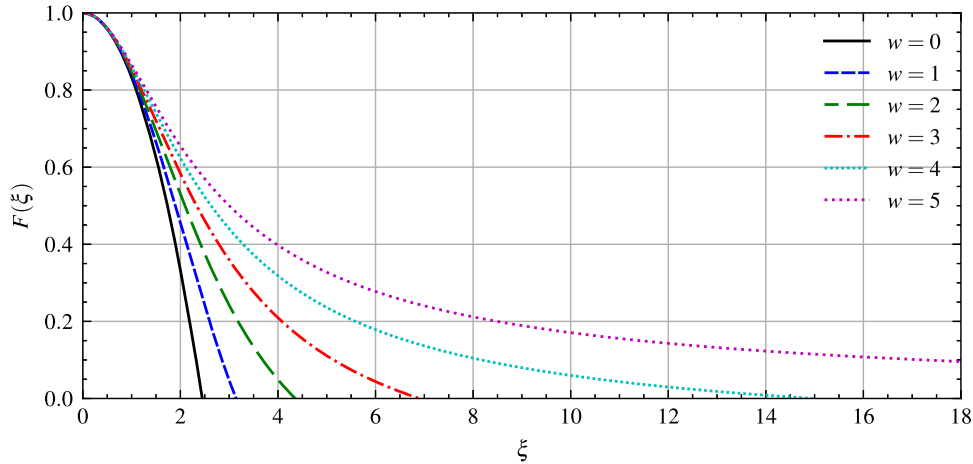


Fig. 9. Approximate solutions to the Lane-Emden differential equation using the proposed rKAN method.

Table 6

Performance of different activation functions in a CNN for classifying the MNIST dataset. It is observed that Jacobi-rKAN(p) with Jacobi polynomial of order p outperforms fKAN in certain cases.

Act. Fun.	MNIST	CIFAR10	CIFAR100	SVHN
Sigmoid	98.09 $\pm$ 0.09	49.60 $\pm$ 0.46	26.18 $\pm$ 0.43	82.76 $\pm$ 0.27
Tanh	98.90 $\pm$ 0.07	65.84 $\pm$ 0.50	38.66 $\pm$ 0.39	84.50 $\pm$ 0.11
ReLU	99.14 $\pm$ 0.04	68.71 $\pm$ 0.51	40.69 $\pm$ 0.53	87.13 $\pm$ 0.24
fKAN	99.23 $\pm$ 0.05	67.27 $\pm$ 0.38	40.79 $\pm$ 0.37	87.89 $\pm$ 0.24
KAN	73.26 $\pm$ 15.11	15.27 $\pm$ 3.11	10.54 $\pm$ 6.97	52.82 $\pm$ 8.01
WavKAN	94.31 $\pm$ 4.11	45.27 $\pm$ 5.11	19.11 $\pm$ 10.50	56.73 $\pm$ 11.19
FastKAN	94.69 $\pm$ 5.11	48.45 $\pm$ 9.59	21.88 $\pm$ 5.14	69.10 $\pm$ 9.00
rKAN(2)	<u>99.27</u> $\pm$ 0.07	67.68 $\pm$ 0.47	40.19 $\pm$ 0.24	88.78 $\pm$ 0.14
rKAN(3)	99.21 $\pm$ 0.05	70.49 $\pm$ 0.86	42.69 $\pm$ 0.55	88.48 $\pm$ 0.16
rKAN(4)	99.06 $\pm$ 0.03	67.13 $\pm$ 0.59	39.63 $\pm$ 0.36	88.61 $\pm$ 0.10
rKAN(5)	99.29 $\pm$ 0.06	71.97 $\pm$ 0.48	43.51 $\pm$ 0.35	88.87 $\pm$ 0.24
rKAN(6)	99.22 $\pm$ 0.09	<u>70.53</u> $\pm$ 0.66	42.28 $\pm$ 0.25	89.17 $\pm$ 0.13

5.1.4. Reinforcement learning

To assess the effectiveness of trainable rational basis functions in control-based environments, we employed a deep Q-Network (DQN) architecture on the CartPole-v1 task. The input state vector has dimension 4, corresponding to the environment's observable features. The policy and target networks share the same architecture: an input layer maps $\mathbb{R}^4 \rightarrow \mathbb{R}^{128}$ through a fully connected layer, followed by a Jacobi-rKAN

module. The second hidden layer maps $\mathbb{R}^{128} \rightarrow \mathbb{R}^{128}$, also followed by an rKAN layer. The output layer is linear, mapping $\mathbb{R}^{128} \rightarrow \mathbb{R}^2$, where each output represents the Q-value of one discrete action. The model is optimized using the AdamW optimizer with a learning rate of 3×10^{-4} , trained over 300 episodes using mini-batches of size 128. Experience replay is performed with a buffer size of 10,000. The exploration strategy is ϵ -greedy with exponential decay from 0.9 to 0.01, and the discount factor is set to $\gamma = 0.99$. The target network parameters are softly updated at each step using the rule $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$, with $\tau = 0.005$. The performance comparison between Jacobi-rKAN and classical activation functions is illustrated in Fig. 8, where the rKAN-based agent reaches the maximum reward of 500 within approximately 200 episodes, whereas ReLU and hyperbolic tangent activations fail to converge within the 300-episode horizon.

5.2. Physics-informed deep learning

Physics-informed deep learning consists of neural network approaches designed to solve differential equations by integrating governing physical laws directly into their loss function. KANs have been developed to address these challenges (Aghaei, 2025; Liu et al., 2025; Nehma & Tiwari, 2024; Shukla et al., 2024; Wang et al., 2024). In these networks, the loss function is defined to enable the network to approximate the dynamics of physical problems. For example, for a differential equation in the operator form $\mathcal{L}(F) = 0$ with initial condition $F(0) = F_0$,

Table 7

A comparison among Jacobi-rKAN, fKAN, and classical activation functions on the IMDB dataset.

Act. Func.	Loss	Accuracy	Precision	Recall	ROC-AUC
Sigmoid	0.604 ± 0.039	85.198 ± 0.468	81.440 ± 1.511	91.266 ± 1.532	91.631 ± 0.422
Tanh	0.610 ± 0.060	85.604 ± 0.818	82.364 ± 2.790	90.912 ± 2.705	91.894 ± 0.505
ReLU	0.808 ± 0.064	83.826 ± 1.141	78.197 ± 2.124	93.995 ± 1.358	91.080 ± 0.404
fKAN	<u>0.559 ± 0.062</u>	85.645 ± 0.762	83.395 ± 3.549	<u>93.190 ± 2.584</u>	92.497 ± 0.404
FastKAN	1.386 ± 0.149	83.532 ± 0.119	83.536 ± 0.101	83.532 ± 1.001	91.287 ± 0.069
WavKAN	0.799 ± 0.025	78.020 ± 0.612	79.240 ± 0.021	78.020 ± 0.020	88.015 ± 1.091
KAN	0.910 ± 0.030	83.992 ± 0.012	84.070 ± 0.015	83.992 ± 0.015	91.038 ± 0.001
rKAN(2)	0.541 ± 0.029	86.598 ± 0.081	86.625 ± 1.477	86.643 ± 2.053	<u>92.500 ± 0.297</u>
rKAN(3)	0.578 ± 0.069	<u>85.727 ± 1.570</u>	<u>83.545 ± 3.966</u>	89.544 ± 3.289	92.519 ± 0.497
rKAN(4)	0.634 ± 0.184	84.103 ± 2.990	80.976 ± 6.669	90.875 ± 5.197	91.961 ± 1.441
rKAN(5)	0.581 ± 0.083	84.974 ± 1.682	80.404 ± 3.074	92.842 ± 1.812	92.307 ± 0.767
rKAN(6)	0.617 ± 0.124	83.759 ± 2.416	79.526 ± 5.803	92.312 ± 4.868	91.947 ± 1.124

Table 8

Comparison of the first roots of the predicted solution with the exact roots from Horedt (2004) and the approximated results from similar neural network approaches (Aghaei, 2025; Liu et al., 2025; Mazraeh & Parand, 2024) as well as a machine learning approach with rational Jacobi polynomial basis function.

w	Jacobi-rKAN	Padé-rKAN	fKAN (Aghaei, 2025)	KAN (Liu et al., 2025)	GEPINN (Mazraeh & Parand, 2024)	LSSVR
0	5.15×10^{-6}	3.18×10^{-6}	3.52×10^{-5}	2.19×10^{-6}	1.40×10^{-7}	3.73×10^{-2}
1	7.12×10^{-5}	8.67×10^{-6}	8.67×10^{-6}	7.78×10^{-5}	4.83×10^{-3}	9.11×10^{-2}
2	2.88×10^{-5}	6.19×10^{-6}	9.34×10^{-6}	9.11×10^{-5}	8.93×10^{-3}	4.14×10^{-2}
3	2.40×10^{-4}	9.11×10^{-6}	5.55×10^{-7}	3.11×10^{-5}	1.88×10^{-2}	1.07×10^{-1}
4	1.57×10^{-3}	8.18×10^{-5}	2.97×10^{-4}	1.27×10^{-3}	5.08×10^{-2}	2.97×10^{-2}
CPU time	1 min	1.5 min	1 min	3 min	180 min	5 min

the network loss is defined as the mean squared residual (Dana Mazraeh et al., 2025):

$$\text{Loss}(\xi) = \frac{1}{|\xi|} \sum_{i=1}^{|\xi|} \mathcal{L}(F)(\xi_i)^2 + |\hat{F}(0) - F_0|^2, \quad (27)$$

where ξ represents the training data in the domain of the problem. For this type of problem, we will focus on the well-known Lane-Emden ordinary differential equation. This equation represents a dimensionless form of Poisson's equation, which describes the gravitational potential of a Newtonian, self-gravitating, spherically symmetric, polytropic fluid. For a positive integer w , this problem is defined as follows:

$$\begin{aligned} \frac{d^2}{d\xi^2} F(\xi) + \frac{2}{\xi} \frac{d}{d\xi} F(\xi) + F^w(\xi) &= 0, \\ F(0) &= 1, \quad F'(0) = 0. \end{aligned} \quad (28)$$

To solve this equation, we use rKANs with both Padé and Jacobi basis functions. We maintain a network architecture consistent with fKAN (Aghaei, 2025) for a fair comparison, but after replacing its basis layer with our rKAN basis. The polynomial order (p, q) of the basis function was tuned for each problem instance (w) using a grid search. Each model was optimized on 1500 equidistant points in the domain $[0, 15]$ using the L-BFGS algorithm, which was run until convergence. Fig. 9 depicts the learned solutions to this problem for different values of w .

The first positive root of the predicted solution to the differential equation holds significant physical meaning. In Table 8, we present a comparison of the absolute error in the first root against several strong baselines, including fractional KAN (fKAN) (Aghaei, 2025), the original spline-based KAN (Liu et al., 2025), the symbolic-regression-based GEPINN (Mazraeh & Parand, 2024), and a classical Least Squares Support Vector Regression (LSSVR) model using a rational Jacobi kernel and quasi-linearization method (Babaei et al., 2024). We also evaluated wav-KAN architecture, but it failed to converge to a meaningful solution Error $> 10^{-1}$, so it is excluded from the table.

The results in Table 8 provide several key insights. While the highly specialized GEPINN method achieves state-of-the-art accuracy, it does so

at a computational cost that is orders of magnitude higher. Among the neural network approaches, the well-tuned Padé-rKAN is now the top performer for $w \in \{1, 2, 4\}$ and is highly competitive for $w = 3$. Importantly, both rKAN variants significantly outperform the original spline-based KAN in accuracy while being 2–3 times faster. They also far exceed the accuracy of the classical LSSVR method, confirming the advantage of the neural network framework. These findings underscore that while rKAN can be sensitive to hyperparameter choices, it is a powerful and efficient architecture for solving complex ODEs.

6. Conclusion

This paper addressed a key limitation in existing Kolmogorov-Arnold networks: the reliance on basis functions like splines or standard polynomials, which struggle with functions exhibiting global behaviors such as singularities and asymptotic decay. Our primary objective was to demonstrate that integrating rational functions as the core learnable basis could overcome these challenges and improve accuracy and parameter efficiency. We successfully demonstrated this by introducing two novel rational KAN architectures, rKANs, based on Padé approximation and rational Jacobi functions.

In the methodology section, we have introduced a new perspective on Kolmogorov-Arnold networks utilizing rational basis functions. Rational functions enhance prediction accuracy, particularly in scenarios involving asymptotic behavior and singularities. We proposed two types of rational KANs based on the Padé approximation and rational Jacobi functions. The first architecture employs the division of two polynomials, specifically shifted Jacobi functions, while the second approach maps Jacobi functions directly into a rational space. In both models, the basis function hyperparameters α , β , and ι (for rational Jacobi functions) are optimized as network weights.

We validated the effectiveness of the proposed method through simulations on real-world examples, including some regression tasks, two classification tasks, and numerical approximations for solving the Lane-Emden ordinary differential equation. The results indicate that the

method can sometimes achieve greater accuracy compared to existing alternatives. However, the experiments showed that the Padé-rKAN increases the time complexity of training, as it involves the computation of two weighted polynomials.

The success of rKANs holds broader implications for deep learning and scientific computing. Our findings suggest that the principled integration of classical approximation theory into modern neural network architectures is a highly fruitful research direction. For scientific machine learning and physics-informed problems, where underlying solutions often have known asymptotic properties, rational-function-based models offer a more natural and efficient function space for approximation. Such architectures could lead to more accurate and robust solutions for solving complex differential equations or modeling physical systems where extrapolation and stability are critical.

For future work, we suggest exploring the use of rational versions of B-spline curves (Bardis & Patrikalakis, 1990; Tiller, 1983), which are renowned for their flexible shape representation capabilities. Furthermore, developing fractional order of rational KANs can be considered as a future direction, particularly for solving physics-informed problems defined on semi-infinite domains (Aghaei et al., 2023; Babaei et al., 2024). As an example of such a basis function, one can consider the algebraic fractional rational mapping $\phi(\xi; i; \gamma) = \{(\text{ReLU}(\xi))^\gamma - i\} / \{(\text{ReLU}(\xi))^\gamma + i\}$.

CRedit authorship contribution statement

Alireza Afzal Aghaei: Writing – original draft; **Mehdi Hosseinzadeh:** Writing – review & editing, Resources; **Kourosh Parand:** Writing – original draft, Project administration.

Data availability

The authors confirm that the data supporting the findings of this study are available through well-known dataset repositories such as Kaggle and UCI Machine Learning Repository.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Abd Elaziz, M., Fares, I. A., & Aseeri, A. O. (2024). Ckan: Convolutional kolmogorov-arnold networks model for intrusion detection in iot environment. *IEEE Access*. <https://doi.org/10.1109/access.2024.3462297>
- Aghaei, A. A. (2025). Fkan: Fractional kolmogorov-arnold networks with trainable jacobi basis functions. *Neurocomputing*, 623, 129414. <https://doi.org/10.1016/j.neucom.2025.129414>
- Aghaei, A. A., Parand, K., Nikkha, A., & Jaber, S. (2023). Solving Falkner-Skan type equations via Legendre and Chebyshev neural blocks. *arXiv:2308.03337*.
- Andrianova, O., Poznyak, A., Fuentes-Aguilar, R. Q., & Chairez, I. (2021). Rational continuous neural network identifier for singular perturbed systems with uncertain dynamical models. *IEEE Transactions on Neural Networks and Learning Systems*, 34(5), 2374–2385. <https://doi.org/10.1109/tnnls.2021.3106574>
- Azam, B., & Akhtar, N. (2024). Suitability of KANs for Computer Vision: A preliminary investigation. *arXiv:2406.09087*.
- Babaei, M., Aghaei, A. A., Kazemi, Z., Jamshidi, M., Ghaderi, R., & Parand, K. (2024). Solving a class of thomas-Fermi equations: A new solution concept based on physics-informed machine learning. *Mathematics and Computers in Simulation*. <https://doi.org/10.1016/j.matcom.2024.06.009>
- Baker Jr, G. A., & Gammel, J. L. (1961). The padé approximant. *Journal of Mathematical Analysis and Applications*, 2(1), 21–30. [https://doi.org/10.1016/0022-247X\(61\)90042-7](https://doi.org/10.1016/0022-247X(61)90042-7)
- Bardis, L., & Patrikalakis, N. M. (1990). Surface approximation with rational B-splines. *Engineering with Computers*, 6, 223–235.
- Biswas, K., Banerjee, S., & Pandey, A. K. (2021). Orthogonal-Padé Activation Functions: Trainable Activation functions for smooth and faster convergence in deep networks. *arXiv:2106.09693*.
- Boullé, N., Nakatsukasa, Y., & Townsend, A. (2020). Rational neural networks. *Advances in Neural Information Processing Systems*, 33, 14243–14253.
- Boyd, J. P. (1992). Defeating the runge phenomenon for equispaced polynomial interpolation via tikhonov regularization. *Applied Mathematics Letters*, 5(6), 57–59. [https://doi.org/10.1016/0893-9659\(92\)90014-z](https://doi.org/10.1016/0893-9659(92)90014-z)
- Boyd, J. P. (2001). Chebyshev and Fourier spectral methods. Courier Corporation.
- Bozorgasl, Z., & Chen, H. (2024). Wav-kan: Wavelet kolmogorov-arnold networks. *arXiv:2405.12832*.
- Braun, J., & Griebel, M. (2009). On a constructive proof of kolmogorov's superposition theorem. *Constructive Approximation*, 30, 653–675. <https://doi.org/10.1007/s00365-009-9054-2>
- Cheon, M. (2024). Kolmogorov-Arnold Network for Satellite Image Classification in Remote Sensing. *arXiv:2406.00600*.
- Clevert, D.-A., Unterthiner, T., & Hochreiter, S. (2015). Fast and accurate deep network learning by exponential linear units (elus). *arXiv:1511.07289*.
- Dana Mazraeh, H., Nosrati Firoozsalari, A., Afzal Aghaei, A., & Parand, K. (2025). A novel neural network architecture for solving fractional differential equations. *Computational Methods for Differential Equations*. <https://doi.org/10.22034/cmde.2025.63059.2803>
- Dzhenezher, S., & Skopenkov, A. (2021). A structured proof of Kolmogorov's Superposition Theorem. *arXiv:2105.00408*.
- Fares, I. A., Abd Elaziz, M., Aseeri, A. O., Zied, H. S., & Abdellatif, A. G. (2025a). TFKAN: Transformer based on kolmogorov-arnold networks for intrusion detection in IoT environment. *Egyptian Informatics Journal*, 30, 100666. <https://doi.org/10.1016/j.eij.2025.100666>
- Fares, I. A., Abdellatif, A. G., Abd Elaziz, M., Shrahili, M., Elmahallawy, A., Sohaib, R. M., Shawky, M. A., & Shah, S. T. (2025b). Deep transfer learning based on hybrid swin transformers with LSTM for intrusion detection systems in IoT environment. *IEEE Open Journal of the Communications Society*. <https://doi.org/10.1109/ojcoms.2025.3569301>
- He, X.-s., He, F., & He, A.-l. (2018). Super-gaussian bss using fast-ica with chebyshev-pade approximant. *Circuits, Systems, and Signal Processing*, 37, 305–341. <https://doi.org/10.1007/s00034-017-0554-1>
- Helmke, U., & Williamson, R. C. (1995). Neural networks, rational functions, and realization theory. *Mathematics of Control, Signals and Systems*, 8, 27–49. <https://doi.org/10.1007/bf01212365>
- Horedt, G. P. (2004). Polytopes: applications in astrophysics and related fields (vol. 306). Springer Science & Business Media.
- Kiamari, M., Kiamari, M., & Krishnamachari, B. (2024). GKAN: Graph Kolmogorov-Arnold Networks. *arXiv:2406.06470*.
- Leung, H., & Haykin, S. (1993). Rational function neural network. *Neural Computation*, 5(6), 928–938. <https://doi.org/10.1162/neco.1993.5.6.928>
- Li, C., Liu, X., Li, W., Wang, C., Liu, H., & Yuan, Y. (2024). U-KAN Makes Strong Backbone for Medical Image Segmentation and Generation. *arXiv:2406.02918*.
- Li, Z. (2024). Kolmogorov-Arnold Networks are Radial Basis Function Networks. *arXiv:2405.06721*.
- Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljatic, M., Hou, T. Y., & Tegmark, M. (2025). KAN: Kolmogorov-arnold networks. In *The thirteenth international conference on learning representations*. <https://openreview.net/forum?id=Ozo7qJ5vZi>
- Mazraeh, H. D., & Parand, K. (2024). GEPINN: An innovative hybrid method for a symbolic solution to the lane-Emden type equation based on grammatical evolution and physics-informed neural networks. *Astronomy and Computing*, (p. 100846). <https://doi.org/10.1016/j.ascom.2024.100846>
- Molina, A., Schramowski, P., & Kersting, K. (2019). Padé activation units: End-to-end learning of flexible activation functions in deep networks. *arXiv:1907.06732*.
- Nehma, G., & Tiwari, M. (2024). Leveraging KANs For Enhanced Deep Koopman Operator Discovery. *arXiv:2406.02875*.
- Rad, J. A., Parand, K., & Chakraverty, S. (2023). Learning with fractional orthogonal kernel classifiers in support vector machines: Theory, algorithms and applications. Springer. <https://doi.org/10.1007/978-981-19-6553-1>
- Schmidt-Hieber, J. (2021). The kolmogorov-arnold representation theorem revisited. *Neural Networks*, 137, 119–126. <https://doi.org/10.1016/j.neunet.2021.01.020>
- Seydi, S. T. (2024). Unveiling the Power of Wavelets: A Wavelet-based Kolmogorov-Arnold Network for Hyperspectral Image Classification. *arXiv:2406.07869*.
- Shen, J., Tang, T., & Wang, L.-L. (2011). Spectral methods: algorithms, analysis and applications (vol. 41). Springer Science & Business Media.
- Shukla, K., Toscano, J. D., Wang, Z., Zou, Z., & Karniadakis, G. E. (2024). A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and operator networks. *arXiv:2406.02917*.
- Somvanshi, S., Javed, S. A., Islam, M. M., Pandit, D., & Das, S. (2024). A Survey on Kolmogorov-Arnold Network. *arXiv:2411.06078*.
- SS, S. (2024). Chebyshev Polynomial-Based Kolmogorov-Arnold Networks: An Efficient Architecture for Nonlinear Function Approximation. *arXiv:2405.07200*.
- Sun, W., & Yang, Z. (2023). Rational function neural networks for learning rock physics models from field data. *Journal of Geophysics and Engineering*, 20(6), 1206–1226. <https://doi.org/10.1093/jge/gxad079>
- Ta, H.-T. (2024). BSRBF-KAN: A combination of B-splines and Radial Basic Functions in Kolmogorov-Arnold Networks. *arXiv:2406.11173*.
- Telgarsky, M. (2017). Neural networks and rational functions. In D. Precup, & Y. W. Teh (Eds.), *Proceedings of the 34th international conference on machine learning* (pp. 3387–3393). PMLR (vol. 70). Proceedings of Machine Learning Research. <https://proceedings.mlr.press/v70/telgarsky17a.html>
- Tiller, W. (1983). Rational B-splines for curve and surface representation. *IEEE Computer Graphics and Applications*, 3(6), 61–69. <https://doi.org/10.1109/mcg.1983.263244>

- Wang, Y., Sun, J., Bai, J., Anitescu, C., Eshaghi, M. S., Zhuang, X., Rabczuk, T., & Liu, Y. (2024). Kolmogorov Arnold Informed neural network: A physics-informed deep learning framework for solving PDEs based on Kolmogorov Arnold Networks, [arXiv:2406.11045](https://arxiv.org/abs/2406.11045).
- Yang, X., & Wang, X. (2024). Kolmogorov-Arnold transformer. [arXiv:2409.10594](https://arxiv.org/abs/2409.10594).
- Yang, Z., Li, D., Nai, W., Liu, L., Sun, J., & Lv, X. (2024). Compressible non-Newtonian fluid based road traffic flow equation solved by physical-Informed rational neural network. *IEEE Access*. <https://doi.org/10.1109/access.2024.3356173>
- Zhangyanbo (2024). MLP-KAN. <https://github.com/Zhangyanbo/MLP-KAN>.